1. Заголовок

Кейс «Система семантического поиска по базе знаний компании» — направлен на проектирование цифрового решения для интеллектуального поиска информации в корпоративных документах.

Направление подготовки: 09.03.03 – Прикладная информатика (или аналогичное)

Образовательная программа: Прикладной искусственный интеллект

Семестр: 2-й семестр

2. Аннотация

Кейс «Система семантического поиска по базе знаний компании» представляет собой задачу по созданию веб-сервиса, который помогает сотрудникам быстро находить нужные инструкции и документы, формулируя запрос на естественном языке, а не только по ключевым словам.

Учебная дисциплина: Ознакомительная практика / Проектный практикум

Актуальность и практическая значимость:

В любой компании накапливаются сотни регламентов и инструкций. Обычный поиск (Ctrl+F) часто не работает, если сотрудник не знает точного названия документа или термина. Семантический поиск («по смыслу») решает эту проблему, позволяя найти документ «Как оформить отпуск», даже если в запросе написано «хочу отдохнуть». Для студентов это отличная возможность познакомиться с основами NLP (обработки естественного языка) и векторыми базами данных на базовом уровне.

Форма выполнения: групповая.

Кейс направлен на формирование следующих ключевых компетенций:

Профессиональные компетенции (hard skills):

- **Основы NLP и ML:** Использование готовых open-source библиотек (например, sentence-transformers, spacy или простых API) для превращения текста в векторное представление (эмбеддинги).
- **Backend-разработка:** Создание простого API (например, на Python/FastAPI или Flask) для приема запросов и выдачи результатов.
- **Работа с данными:** Генерация синтетического набора данных (текстов инструкций) и их хранение (простая БД или файловая система).
- **Frontend (Базовый):** Создание минималистичного интерфейса (поисковая строка и список выдачи).
- **DevOps (Основы):** Работа с Git (Сфера.Код), базовый деплой на бесплатный хостинг.

Универсальные компетенции (soft skills):

- **Командная работа:** Распределение ролей (кто делает интерфейс, кто настраивает логику поиска).
- **Управление проектом:** Планирование спринтов в трекере (Сфера.Задачи), соблюдение дедлайнов.
- **Системное мышление:** Понимание того, как запрос пользователя превращается в ответ системы.

3. Постановка задания

Название кейса: Система семантического поиска по базе знаний компании (MVP).

Цель выполнения кейса:

Создать работающий прототип поисковой системы, который умеет индексировать небольшой набор текстовых документов и находить наиболее подходящие из них по смысловой близости к запросу пользователя.

Конкретный образ результата:

Веб-сервис, развернутый на публичном хостинге (или доступный локально при защите, если хостинг невозможен), который позволяет:

1. **Администратору:** Загружать текстовые документы (статьи, инструкции) в базу знаний.
2. **Пользователю:** Вводить поисковый запрос в свободной форме (например, «что делать если сломался принтер»).
3. **Системе:** Выводить список документов, отсортированных по релевантности (смысловому сходству).

Важное упрощение: Студентам не нужно обучать нейросети с нуля. Достаточно использовать готовые предобученные модели для векторизации текста.

Промежуточные показы (Milestones):

- **Показ 1 (Конец 3-й недели):** Защита концепции и данных. Студенты показывают сгенерированную базу знаний (синтетические данные) и выбранный стек технологий.
- **Показ 2 (Конец 6-й недели):** Демонстрация «Админки». Реализована загрузка и сохранение текстов в базу данных.
- **Показ 3 (Конец 9-й недели):** Демонстрация «Мозгов». Реализован скрипт, который находит нужный текст по запросу (без красивого интерфейса, можно в консоли/Swagger).
- **Показ 4 (Конец 12-й недели):** Демонстрация MVP. Интерфейс поиска соединен с логикой.

Форма и формат отчетности (финальная):

1. **Ссылка на работающий сервис** (или демонстрация экрана с локальной машины).
2. **Ссылка на репозиторий в Сфера.Код** (или аналоге).
3. **Набор тестовых данных** (файл с инструкциями, который использовался для проверки).
4. **Финальная презентация**.

Сроки выполнения: 15 недель.

4. Рекомендуемые инструменты

Таск-трекер Сфера.Задачи для настройки доски, постановки задач с ролями по спринтам и фиксации дефектов.

Wiki-система Сфера.Знания для формирования и хранения технической документации.

Git-репозиторий Сфера.Код для создания, версионирования и хранения кода.

5. Методические рекомендации обучающимся

Рекомендуемый состав группы по ролям (3-4 человека):

- **Team Lead / Analyst:** Генерирует датасет (используя ChatGPT/GigaChat), пишет требования, следит за задачами в Сфере.
- **ML/Backend Engineer:** Интегрирует библиотеку поиска (превращает текст в векторы), пишет логику сравнения.
- **Fullstack Developer:** Делает простой API для загрузки файлов и страницу поиска.

Примерный план работы по спринтам (3 недели на спринт):

Спринт 1 (Недели 1-3): Данные и Исследование

- **Задачи:**
  - Изучить, что такое «векторный поиск» и «косинусное сходство» (на базовом уровне).
  - Сгенерировать с помощью ИИ (GigaChat, ChatGPT и др.) 20-30 текстовых документов для «вымышленной компании» (Инструкция по пожарной безопасности, Как оформить командировку, Правила дресс-кода и т.д.).
  - Выбрать библиотеку для NLP (рекомендуется sentence-transformers для Python или аналоги).
  - Завести проект в Сфера.Задачи и репозиторий в Сфера.Код.

- **Результат:** Готовый текстовый файл с данными (JSON/CSV) и понимание, какую библиотеку использовать.

Спринт 2 (Недели 4-6): Хранение документов (Бэкенд)

- **Задачи:**
  - Развернуть базовый бэкенд (Python/Node.js).
  - Сделать простую базу данных (можно использовать SQLite или даже JSON-файл для упрощения).
  - Реализовать API метод POST /add-document, который принимает текст и заголовок и сохраняет их.
  - (Опционально) Сделать простую форму загрузки документов.
- **Результат:** Можно отправить текст на сервер, и он сохраняется в системе.

Спринт 3 (Недели 7-9): Логика поиска (Ядро)

- **Задачи:**
  - Реализовать функцию векторизации: при сохранении документа превращать его текст в вектор чисел (используя выбранную в 1 спринте библиотеку).
  - Реализовать API метод GET /search. Он должен:
    1. Принять запрос пользователя.
    2. Превратить запрос в вектор.
    3. Сравнить вектор запроса с векторами документов (найти ближайшие).
    4. Вернуть топ-3 документа.
- **Результат:** Работающий поиск через Swagger/Postman или консоль. Вводишь «отпуск» — получаешь документ «Правила отдыха».

Спринт 4 (Недели 10-12): Интерфейс (UI) и Интеграция

- **Задачи:**
  - Сделать простую веб-страницу с поисковой строкой.
  - Отображать результаты поиска в виде карточек (Заголовок + фрагмент текста).
  - Соединить страницу с API поиска.
- **Результат:** MVP, где пользователь может ввести вопрос в браузере и увидеть ответ.

Спринт 5 (Недели 13-15): Тестирование и Документация

- **Задачи:**
  - Проверить поиск на хитрых запросах (синонимы, опечатки).
  - Написать инструкцию (README.md): как запустить проект, какие библиотеки нужны.
  - Подготовить презентацию: какую проблему решали, какие технологии использовали, примеры работы.
- **Результат:** Готовый проект к защите.

6. Методические рекомендации по оцениванию

Критерии оценивания (максимум 100 баллов):

Блок 1. Промежуточные показы (40 баллов)

- **Показ 1 (10 баллов):** Есть качественный набор синтетических данных (не мусорный текст) и обоснован выбор инструмента поиска.
- **Показ 2 (10 баллов):** Работает сохранение документов в базу. Код в репозитории.
- **Показ 3 (10 баллов):** Самый важный этап. Поиск реально работает и находит документы по смыслу (а не просто по точному совпадению слов).
- **Показ 4 (10 баллов):** Есть работающий интерфейс в браузере.

Блок 2. Финальный продукт (40 баллов)

- **Качество поиска (20 баллов):** Система находит релевантные документы. Например, на запрос «где поесть» находит документ «Расположение столовой», хотя слова «поесть» там нет.
- **Простота развертывания (10 баллов):** Проект можно легко запустить, следуя инструкции.
- **Интерфейс (10 баллов):** Аккуратный, минималистичный, понятный UI.

Блок 3. Защита (20 баллов)

- **Презентация (10 баллов):** Понятный рассказ, демонстрация в живом режиме.
- **Ответы на вопросы (10 баллов):** Команда понимает, как работает их алгоритм поиска (на базовом уровне понимает, что такое эмбеддинги/вектора).

Описание уровней оценки:

- **Отлично:** Поиск работает семантически (понимает смысл). Интерфейс приятный. Код чистый.
- **Хорошо:** Поиск работает, но иногда сбоит или работает как обычный поиск по ключевым словам. Проект запускается.
- **Удовлетворительно:** Реализован только поиск по точному совпадению слов. Интерфейс «сырой».
- **Неудовлетворительно:** Проект не работает, поиск не реализован.